```
#!/usr/bin/env python3
"""
THE SYNDICATE: GLOBAL COMPENSATION MATRIX & EQUILIBRIUM ENGINE (ACT OF 2026)
================================================================================
Open-Source Reference Implementation of the Calibrated Thermodynamic,
Full-Lifecycle (LCA), and Temporal Irreversibility Accounting Standard.

Core Philosophy:
    "Our goal is not to coerce, punish, or subjugate, but to prove—through
    rigorous mathematics and indisputable physics—that mindful consumption
    and the rational stewardship of resources are the only reliable keys
    to human prosperity."

Mathematical Core:
    1. Extended Dynamic Equilibrium Formula (I_ndp Index):
       Tariff ($) = B_r * [ Sum(M * D_f * E_v * L_c) * T_ir ] * M_f

    2. Universal Planetary Impact Equation (UPIE-2026 Batch/Capacity Mode):
       Phi ($) = B_r * [ (S_infra * E_d * L_c_infra) +
                         Sum(Q_i * m_i * D_f_i * E_v * L_c * T_ir) ] * M_f

    3. Temporal Irreversibility & Recovery Deficit Coefficient (T_ir):
       T_ir = 1.0 + I_geo + C_bio + ln(1 + tau_recovery / tau_emission)

License: MIT / Open Equilibrium Standard (Category Alpha Public Domain)
"""

from __future__ import annotations
import json
import math
from dataclasses import dataclass, field, asdict
from enum import Enum
from typing import List, Dict, Optional


#
================================================================================
# 1. GLOBAL PHYSICAL & ECONOMIC CONSTANTS
#
```

```
================================================================================

# Base Rate (B_r): Anchored to the physical cost of capturing/mineralizing
# 1 metric ton (1,000 kg) of CO2, detoxifying 100 m3 of marine water,
# or restoring an equivalent stratospheric O3 column.
BASE_RATE_USD_PER_TON_POINT: float = 10.0
BASE_RATE_USD_PER_KG_POINT: float = 0.01


class EntityCategory(str, Enum):
    """Binary categorization under Article 3 of the Charter."""
    ALPHA = "Category Alpha (25-50 Yr Durable, Circular & Scientific Units)"
    OMEGA = "Category Omega (Planned Obsolescence, Subsidized & Coercive Units)"


class SupplyChainTier(str, Enum):
    """Industrial role classification for Temporal Irreversibility (T_ir)."""
    ALPHA_CIRCULAR = "Category Alpha Circular / Biodegradable (T_ir = 1.0)"
    TIER_1_EXTRACTOR = "Tier I: Primary Subsurface Extractor (T_ir = 3.5 - 5.0)"
    TIER_2_REFINER = "Tier II: Petrochemical & PFAS Refiner (T_ir = 2.5 - 4.2)"
    TIER_3_MANUFACTURER = "Tier III: Obsolescence Manufacturer (T_ir = 1.8 - 3.0)"


#
================================================================================
# 2. TEMPORAL IRREVERSIBILITY & RECOVERY DEFICIT MODEL (T_ir)
#
================================================================================

@dataclass
class TemporalIrreversibilityModel:
    """
    Calculates or stores the Temporal Irreversibility, Bio-Accumulation &
    Recovery Deficit Coefficient (T_ir).

    Formula:
        T_ir = 1.0 + I_geo + C_bio + ln(1 + tau_recovery / tau_emission)
    """
    i_geo: float = 0.0          # Geological/aquifer irreversibility (0.0 to 2.0)
    c_bio: float = 0.0          # Delayed bio-accumulation & cancer latency (0.0 to 1.5)
```

```
    recovery_deficit_term: float = 0.0  # ln(1 + tau_recovery / tau_emission)

    @classmethod
    def from_recovery_times(
        cls,
        i_geo: float,
        c_bio: float,
        tau_recovery_years: float,
        tau_emission_years: float = 1.0
    ) -> TemporalIrreversibilityModel:
        """Derives the recovery deficit dynamically from physical timescales."""
        deficit = math.log(1.0 + (tau_recovery_years / max(tau_emission_years, 1e-6)))
        return cls(i_geo=i_geo, c_bio=c_bio, recovery_deficit_term=deficit)

    @property
    def coefficient(self) -> float:
        """Returns the total calibrated T_ir multiplier (minimum baseline = 1.0)."""
        return round(1.0 + self.i_geo + self.c_bio + self.recovery_deficit_term, 4)


#
================================================================================
====
# 3. CORE DATA STRUCTURES FOR PHYSICAL STREAMS & INFRASTRUCTURE
#
================================================================================
====

@dataclass
class EmissionStream:
    """
    Represents a distinct physical mass flow (M), toxicity factor (D_f),
    and ecosystem vulnerability factor (E_v) within an industrial operation.
    """
    name: str
    mass_tons: float            # Physical mass flow M (in metric tons)
    toxicity_df: float          # Specific compound hazard coefficient (D_f)
    vulnerability_ev: float     # Ecosystem dispersion/vulnerability factor (E_v)
    stream_lc_override: Optional[float] = None  # Optional per-stream L_c

    @property
    def base_ton_points(self) -> float:
        """Calculates unadjusted physical damage points: M * D_f * E_v."""
```

```
        return self.mass_tons * self.toxicity_df * self.vulnerability_ev

    def adjusted_ton_points(self, default_lc: float) -> float:
        """Calculates lifecycle-adjusted damage points: M * D_f * E_v * L_c."""
        lc = self.stream_lc_override if self.stream_lc_override is not None else default_lc
        return self.base_ton_points * lc


@dataclass
class InfrastructureBlock:
    """Represents physical facility footprint debt for UPIE-2026 calculations."""
    area_km2: float             # Summed facility area (S_infra in km2)
    energy_density_ed: float    # Power draw density index (E_d)
    lifecycle_lc_infra: float   # Structural material lifecycle factor (L_c_infra)

    @property
    def infra_ton_points(self) -> float:
        """Calculates S_infra * E_d * L_c_infra."""
        return self.area_km2 * self.energy_density_ed * self.lifecycle_lc_infra


@dataclass
class AuditDossier:
    """
    Complete full-lifecycle audit model for any corporate entity, vessel,
    launch system, or industrial sector.
    """
    dossier_id: int
    name: str
    sector: str
    evaluation_unit: str
    category: EntityCategory
    supply_chain_tier: SupplyChainTier
    streams: List[EmissionStream]
    lifecycle_lc: float
    manipulation_mf: float
    temporal_model: TemporalIrreversibilityModel = field(
        default_factory=TemporalIrreversibilityModel
    )
    infrastructure: Optional[InfrastructureBlock] = None
    one_time_capital_debt_usd: float = 0.0
    annual_units_multiplier: float = 8760.0  # Default: 8,760 hours/year
    historical_units_multiplier: float = 0.0
```

```
reported_annual_profit_usd: Optional[float] = None
conventional_fines_paid_usd: str = "$0"
hypocrisy_rationale: str = ""
renewal_pact_solution: str = ""

@property
def unadjusted_physical_ton_points(self) -> float:
    """Sum of M * D_f * E_v across all streams (+ infrastructure if applicable)."""
    stream_pts = sum(s.base_ton_points for s in self.streams)
    infra_pts = self.infrastructure.infra_ton_points if self.infrastructure else 0.0
    return stream_pts + infra_pts

@property
def lifecycle_adjusted_ton_points(self) -> float:
    """Sum of (M * D_f * E_v * L_c) across all streams + infrastructure."""
    stream_lc_pts = sum(s.adjusted_ton_points(self.lifecycle_lc) for s in self.streams)
    infra_pts = self.infrastructure.infra_ton_points if self.infrastructure else 0.0
    return stream_lc_pts + infra_pts

@property
def temporal_adjusted_ton_points(self) -> float:
    """Applies Temporal Irreversibility (T_ir) to lifecycle-adjusted points."""
    return self.lifecycle_adjusted_ton_points * self.temporal_model.coefficient

@property
def final_syndicate_ton_points(self) -> float:
    """Applies the Hypocrisy & Manipulation Multiplier (M_f)."""
    return self.temporal_adjusted_ton_points * self.manipulation_mf

@property
def unit_tariff_usd(self) -> float:
    """Final compensation tariff ($) per evaluation unit (hour/launch/batch)."""
    return round(self.final_syndicate_ton_points * BASE_RATE_USD_PER_TON_POINT, 2)

@property
def annual_liability_usd(self) -> float:
    """Projected annual compensation liability ($/year)."""
    return round(self.unit_tariff_usd * self.annual_units_multiplier, 2)

@property
def cumulative_historical_debt_usd(self) -> float:
    """Cumulative historical planetary liability ($) from inception to 2026."""
    return round(self.unit_tariff_usd * self.historical_units_multiplier, 2)
```

```
    def simulate_renewal_pact(self) -> Dict[str, float]:
        """
        Simulates signing The Syndicate Technological Renewal Pact (Article 11):
        - 100% of historical ecological debt is forgiven ($0).
        - Hypocrisy/Obsolescence multiplier M_f drops to 1.0 (honest 25-50 yr engineering).
        - Temporal Irreversibility T_ir drops to 1.0 (closed-loop recovery).
        - Clean energy/process transition reduces physical base emissions by 80%.
        """
        clean_physical_factor = 0.20  # 80% physical emission reduction via escrow grants
        post_pact_unit_ton_pts = self.lifecycle_adjusted_ton_points * clean_physical_factor
        post_pact_unit_usd = round(post_pact_unit_ton_pts *
BASE_RATE_USD_PER_TON_POINT, 2)
        post_pact_annual_usd = round(post_pact_unit_usd * self.annual_units_multiplier, 2)
        savings_pct = (
            (1.0 - (post_pact_unit_usd / self.unit_tariff_usd)) * 100.0
            if self.unit_tariff_usd > 0 else 0.0
        )
        return {
            "pre_pact_unit_tariff_usd": self.unit_tariff_usd,
            "post_pact_unit_tariff_usd": post_pact_unit_usd,
            "pre_pact_annual_liability_usd": self.annual_liability_usd,
            "post_pact_annual_liability_usd": post_pact_annual_usd,
            "forgiven_historical_debt_usd": self.cumulative_historical_debt_usd,
            "tariff_reduction_percentage": round(savings_pct, 2),
        }


#
================================================================================
====
# 4. DIGITAL FOOTPRINT ACCOUNTING ENGINE (ARTICLE 7)
#
================================================================================
====

@dataclass
class DigitalOperationSpec:
    name: str
    power_wh: float
    water_ml: float
    damage_points_kg_eq: float
```

```
    def calculate_tariff(self, count: int = 1, is_single_vector_learning: bool = False) -> float:
        """ Category Alpha educational/scientific queries pay $0.00 tariff. """
        if is_single_vector_learning:
            return 0.0
        return round(count * self.damage_points_kg_eq * BASE_RATE_USD_PER_KG_POINT, 6)


DIGITAL_OPERATIONS_REGISTRY: Dict[str, DigitalOperationSpec] = {
    "standard_search": DigitalOperationSpec("1 Standard Search Query", 0.3, 4.0, 0.03),
    "llm_inference": DigitalOperationSpec("1 Advanced AI Query (LLM)", 3.0, 50.0, 0.45),
    "pdf_compile": DigitalOperationSpec("Compilation of 1 PDF Page", 0.1, 1.0, 0.015),
    "image_gen": DigitalOperationSpec("AI Image Generation", 5.0, 75.0, 0.95),
    "video_gen_1min": DigitalOperationSpec("1 Min AI Video Generation (Sora Class)", 120.0,
1800.0, 25.0),
}


#
================================================================================
====
# 5. CANONICAL REGISTRY OF THE 9 SECTOR & CORPORATE AUDIT DOSSIERS
#
================================================================================
====

def build_canonical_dossiers() -> List[AuditDossier]:
    """
    Instantiates all 9 calibrated corporate and sector dossiers exactly as
    defined in Section III of The Syndicate Manifesto.
    """
    dossiers: List[AuditDossier] = []

    # ---------------------------------------------------------------------
    # DOSSIER 1: SHIPBUILDING & HFO MARITIME VESSEL OWNERSHIP
    # ---------------------------------------------------------------------
    # Part A Capital Hull Debt: 125,000t * 4.0 (D_f) * 1.0 (E_v) * 2.4 (L_c) * $10 = $12,000,000
    hull_capital_debt = 125_000.0 * 4.0 * 1.0 * 2.4 * BASE_RATE_USD_PER_TON_POINT

    dossiers.append(AuditDossier(
        dossier_id=1,
        name="Legacy HFO Container Vessel (Shipbuilding & Transit)",
        sector="Maritime Logistics & Heavy Shipbuilding",
        evaluation_unit="1 Hour of Open-Ocean HFO Transit (+ $12M Hull Capital Debt)",
```

```
        category=EntityCategory.OMEGA,
        supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
        streams=[
            EmissionStream(
                name="Heavy Fuel Oil (HFO) Bunker Exhaust (SOx, NOx, V/Ni Soot)",
                mass_tons=5.0,
                toxicity_df=54.0,
                vulnerability_ev=5.0  # 2,000 km2 open-ocean plankton habitat plume
            )
        ],
        lifecycle_lc=1.8,
        manipulation_mf=5.0,
        one_time_capital_debt_usd=hull_capital_debt,
        annual_units_multiplier=6000.0,      # ~250 active sea transit days/year
        historical_units_multiplier=120000.0,
        reported_annual_profit_usd=16_400_000.0,  # ~Charter revenue baseline per vessel
        conventional_fines_paid_usd="~$0 - $5M per fleet (minor port sulfur fines)",
        hypocrisy_rationale=(
            "Burning toxic refinery sludge in international waters to transport "
            "short-lived plastic toys and fast-fashion marketplace cargo."
        ),
        renewal_pact_solution=(
            "Unlock CBDC escrow grants to retrofit propulsion to green
methanol/ammonia/wind-assist "
            "and prioritize durable, 25-year cargo."
        )
    ))

    # ---------------------------------------------------------------------
    # DOSSIER 2: TESLA INC. (GIGAFACTORY + BATTERY MINING + EGO ENGINEERING)
    # ---------------------------------------------------------------------
    dossiers.append(AuditDossier(
        dossier_id=2,
        name="Tesla Inc. (Giga Texas Benchmark & Global EV Fleet)",
        sector="Automotive & Lithium-Nickel-Cobalt Battery Manufacturing",
        evaluation_unit="1 Hour of Peak Gigafactory Operation (~50 EVs/hr + Mineral Debt)",
        category=EntityCategory.OMEGA,
        supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
        streams=[
            EmissionStream(
                name="Induced Regional Power Grid CO2 & Paint-Shop VOCs",
                mass_tons=48.0,
                toxicity_df=2.5,
```

```
            vulnerability_ev=2.0  # 120 km2 Austin metro airshed
        ),
        EmissionStream(
            name="Upstream Battery Mineral Leaching (Lithium Brine, DRC Cobalt, Open-Pit
Nickel)",
            mass_tons=25.0,
            toxicity_df=85.0,
            vulnerability_ev=3.0  # Fragile arid salt flats & tropical watersheds
        )
    ],
    lifecycle_lc=2.8,
    manipulation_mf=3.5,
    annual_units_multiplier=16800.0,      # 4 Global Gigafactories combined active hours/yr
    historical_units_multiplier=210000.0, # 2008-2026 cumulative active hours (>10M EVs)
    reported_annual_profit_usd=12_500_000_000.0,
    conventional_fines_paid_usd="~$1.5M - $1.7M (while collecting billions in green credits)",
    hypocrisy_rationale=(
        "Abandoned affordable humanitarian transit promises to build 3-ton luxury "
        "status trucks and absurd cold-gas thruster 'jumping' sports cars for executive ego."
    ),
    renewal_pact_solution=(
        "End populist ego-vehicles, deploy 100% closed-loop battery recycling, "
        "and pivot Gigafactories to modular public/utility transit."
    )
))

# ---------------------------------------------------------------------
# DOSSIER 3: APPLE INC. (CONSUMER ELECTRONICS & PLANNED OBSOLESCENCE)
# ---------------------------------------------------------------------
dossiers.append(AuditDossier(
    dossier_id=3,
    name="Apple Inc. (Zhengzhou Assembly Hub & 3nm Wafer Supply Chain)",
    sector="Consumer Electronics & Semiconductor Fabrication",
    evaluation_unit="1 Hour of Hub Production (~18,000 Devices/hr + Upstream
PFAS/Silicon)",
    category=EntityCategory.OMEGA,
    supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
    streams=[
        EmissionStream(
            name="Coal-Reliant Industrial Grid Plume (Thermoforming & Cleanrooms)",
            mass_tons=144.0,
            toxicity_df=2.0,
            vulnerability_ev=3.0  # 240 km2 industrial containment dome
```

```
        ),
        EmissionStream(
            name="3nm Silicon Etching Acids, PFAS Forever Chemicals & Rare-Earth Tailings",
            mass_tons=15.0,
            toxicity_df=180.0,
            vulnerability_ev=3.0
        )
    ],
    lifecycle_lc=2.7,
    manipulation_mf=4.5,
    annual_units_multiplier=85000.0,      # Global distributed supply-chain hub hours/yr
    historical_units_multiplier=350000.0, # 1976-2026 (>4.2 Billion devices produced)
    reported_annual_profit_usd=95_000_000_000.0,
    conventional_fines_paid_usd="$0 in direct eco-fines (relies on commercial tree offsets)",
    hypocrisy_rationale=(
        "Greenwashed 'Mother Nature' PR masking monolithic glued batteries, "
        "serialized parts blocking repair, and engineered 24-month obsolescence."
    ),
    renewal_pact_solution=(
        "Re-engineer assembly lines for open-architecture, modular, home-repairable "
        "hardware with a mandatory 25-to-50-year operational lifespan."
    )
))

# ---------------------------------------------------------------------------
# DOSSIER 4: SPACEX (STARSHIP SUPER HEAVY & FALCON LAUNCH FLEET)
# ---------------------------------------------------------------------------
# Note: Historical debt includes 14 Starship launches + 260 non-Starlink Falcon launches
# ($2,743,372,800 + $2,471,040,000 = $5,214,412,800 -> ~26.61 Starship-equivalent units)
dossiers.append(AuditDossier(
    dossier_id=4,
    name="SpaceX Corp. (Starship Super Heavy Launch Vehicle)",
    sector="Heavy Aerospace Launch Logistics",
    evaluation_unit="1 Single Starship Super Heavy Launch Event (0-120s Ascent Corridor)",
    category=EntityCategory.OMEGA,
    supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
    streams=[
        EmissionStream(
            name="Lower Tropospheric Ascent Plume & Coastal Wetland Acoustic Shock",
            mass_tons=2700.0,
            toxicity_df=2.0,
            vulnerability_ev=2.0
        ),
```

```
            EmissionStream(
                name="Stratospheric/Mesospheric Water Vapor & Thermal NOx Injection (PSC Ozone
Depletion)",
                mass_tons=900.0,
                toxicity_df=150.0,
                vulnerability_ev=10.0  # 10,000 km2 hyper-dry stratospheric corridor
            )
        ],
        lifecycle_lc=3.2,
        manipulation_mf=4.5,
        annual_units_multiplier=25.0,          # Projected commercial Starship launch cadence/yr
        historical_units_multiplier=26.61023,  # Exact equivalent for 14 Starships + 260 Falcons
($5.214B)
        reported_annual_profit_usd=3_000_000_000.0,
        conventional_fines_paid_usd="~$152k - $785k (minor EPA/FAA administrative penalties)",
        hypocrisy_rationale=(
            "Destructive test-by-explosion cadence in coastal wildlife reserves and "
            "stratospheric ozone disruption for commercial monopoly saturation."
        ),
        renewal_pact_solution=(
            "Co-develop closed-loop zero-toxicity launch propulsion and electromagnetic "
            "mass-drivers dedicated to international deep-space science."
        )
    ))

    # ---------------------------------------------------------------------
    # DOSSIER 5: STARLINK (ORBITAL INCINERATION & GEOPOLITICAL COERCION)
    # ---------------------------------------------------------------------
    dossiers.append(AuditDossier(
        dossier_id=5,
        name="Starlink Megaconstellation (5-Yr Disposable Satellites)",
        sector="Orbital Telecommunications & Aerospace Infrastructure",
        evaluation_unit="1 Full Year of Constellation Maintenance (70 Launches + 800t Sky Burn)",
        category=EntityCategory.OMEGA,
        supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
        streams=[
            EmissionStream(
                name="Annual Orbital Delivery (70 Falcon 9 Upper-Stage Stratospheric Soot
Plumes)",
                mass_tons=8400.0,       # 70 launches * 120 tons upper-stage RP-1 kerosene
                toxicity_df=120.0,
                vulnerability_ev=10.0,
                stream_lc_override=2.2
```

```
        ),
        EmissionStream(
            name="The Sky Crematorium (1,500 Expired Satellites/Yr Vaporized into Al2O3
Nanoparticles)",
            mass_tons=800.0,        # 800 metric tons of aluminum, lithium & copper burned at
70km
            toxicity_df=2500.0,
            vulnerability_ev=10.0,
            stream_lc_override=3.5
        ),
        EmissionStream(
            name="Ground Segment (5,000,000 Phased-Array Terminals Continuous 3.28 TWh
Grid Draw)",
            mass_tons=1_500_000.0,
            toxicity_df=2.5,
            vulnerability_ev=1.0,
            stream_lc_override=2.7
        )
    ],
    lifecycle_lc=3.0,
    manipulation_mf=6.0,
    annual_units_multiplier=1.0,          # Evaluation unit is already 1 Full Year
    historical_units_multiplier=6.26452,  # 2019-2026 cumulative + committed deorbit debt
($38.45B)
    reported_annual_profit_usd=2_000_000_000.0,
    conventional_fines_paid_usd="$0 (zero regulation on vaporizing 800t/yr of Al2O3 in
mesosphere)",
    hypocrisy_rationale=(
        "Engineered 5-year disposable satellites that use the upper atmosphere as a free "
        "incinerator, while weaponizing humanitarian connectivity as a private geopolitical "
        "switch in active war zones (e.g., disabling coverage during offensive operations in
Ukraine)."
    ),
    renewal_pact_solution=(
        "Extend satellite lifespans to 20-25 years via orbital servicing, eliminate Al2O3 "
        "vaporization, and place network governance under a neutral international humanitarian
protocol."
    )
))

# ---------------------------------------------------------------------
# DOSSIER 6: ULTRA-LARGE DATA CENTER COMPLEXES (600,000 m2, 650 MW)
# ---------------------------------------------------------------------
```

```
    dossiers.append(AuditDossier(
        dossier_id=6,
        name="Ultra-Large Hyperscale Data Center Complex (600,000 m2 / 650 MW)",
        sector="Cloud Computing, Ad-Tech & AI Infrastructure",
        evaluation_unit="1 Hour of Continuous 650 MW Facility Operation",
        category=EntityCategory.OMEGA,
        supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
        streams=[
            EmissionStream(
                name="Continuous 650 MW Grid Carbon Plume & Biocide Cooling Water
Evaporation",
                mass_tons=312.0,
                toxicity_df=2.0,
                vulnerability_ev=2.5  # 50 km2 thermal and aquifer depression zone
            ),
            EmissionStream(
                name="3-Year GPU & Server Hardware Obsolescence Amortization (Silicon, Gold,
PFAS)",
                mass_tons=17.2,
                toxicity_df=100.0,
                vulnerability_ev=2.0
            )
        ],
        lifecycle_lc=2.0,
        manipulation_mf=4.5,
        annual_units_multiplier=8760.0,
        historical_units_multiplier=43800.0,
        reported_annual_profit_usd=4_500_000_000.0,
        conventional_fines_paid_usd="$0 (receives municipal tax breaks for water & power draw)",
        hypocrisy_rationale=(
            "Over 85% of global compute is dedicated to behavioral ad-targeting, dopamine "
            "retention traps, financial speculation, and synthetic spam rather than science."
        ),
        renewal_pact_solution=(
            "Transition to closed-loop geothermal/SMR micro-grids, modular 10-year server "
            "architectures, and dedicate >=80% of compute to science, medicine, and climate
modeling."
        )
    ))

    # ---------------------------------------------------------------------
    # DOSSIER 7: CHEMICAL INDUSTRY BENCHMARK (1 km2 SYNTHESIS COMPLEX)
    # ---------------------------------------------------------------------
```

```
dossiers.append(AuditDossier(
    dossier_id=7,
    name="Averaged Heavy Chemical Synthesis Complex (1 km2 Benchmark)",
    sector="Industrial Chemicals, Fertilizers & Base Polymers",
    evaluation_unit="1 Hour of Steady-State 250 MW Chemical Plant Operation",
    category=EntityCategory.OMEGA,
    supply_chain_tier=SupplyChainTier.TIER_2_REFINER,
    streams=[
        EmissionStream(
            name="Thermal CO2, Fugitive VOCs, NOx & Catalytic Chemical Effluents",
            mass_tons=180.0,
            toxicity_df=15.0,
            vulnerability_ev=3.0  # Soil degradation & regional river watershed
        )
    ],
    lifecycle_lc=3.5,
    manipulation_mf=2.0,  # Balanced utility: produces essential fertilizers/reagents + plastics
    annual_units_multiplier=8760.0,
    historical_units_multiplier=175200.0,
    reported_annual_profit_usd=5_200_000_000.0,
    conventional_fines_paid_usd="~$5M - $20M (routine regulatory permit settlements)",
    hypocrisy_rationale=(
        "Baseline industrial benchmark: demonstrates that an entire 1 km2 heavy chemical "
        "plant costs less per hour ($567k) than Apple's obsolescence hub ($1.089M)."
    ),
    renewal_pact_solution=(
        "Install closed-loop molecular plasma scrubbers, zero-liquid-discharge water loops, "
        "and phase out single-use packaging polymers."
    )
))

# ---------------------------------------------------------------------------
# DOSSIER 8: FAST-FASHION & LOW-QUALITY CLOTHING MANUFACTURING
# ---------------------------------------------------------------------------
dossiers.append(AuditDossier(
    dossier_id=8,
    name="Fast-Fashion Conglomerate (100-Million Garment Seasonal Batch)",
    sector="Apparel, Petrochemical Textiles & Ultra-Fast Marketplaces",
    evaluation_unit="1 Production Batch of 100,000,000 Disposable Synthetic Garments",
    category=EntityCategory.OMEGA,
    supply_chain_tier=SupplyChainTier.TIER_3_MANUFACTURER,
    infrastructure=InfrastructureBlock(
        area_km2=2.0,
```

```
            energy_density_ed=500.0,
            lifecycle_lc_infra=2.5
        ),
        streams=[
            EmissionStream(
                name="Virgin Polyester (80,000t), Carcinogenic Azo Dyes & River Microplastic
Shedding",
                mass_tons=80_000.0,
                toxicity_df=45.0,
                vulnerability_ev=3.0  # Freshwater deltas & open-air textile dumps
            )
        ],
        lifecycle_lc=2.2,
        manipulation_mf=7.5,
        annual_units_multiplier=4.0,         # 4 seasonal batches per year (400M garments/yr)
        historical_units_multiplier=40.0,
        reported_annual_profit_usd=6_000_000_000.0,
        conventional_fines_paid_usd="$0 (river pollution & landfilling externalized overseas)",
        hypocrisy_rationale=(
            " Intentional weak seams, toxic synthetic fabrics, and micro-trend manipulation "
            "designed to turn petroleum and clean river water into landfill waste within 90 days."
        ),
        renewal_pact_solution=(
            "Replace virgin polyester and azo dyes with durable, natural, non-toxic fibers "
            "engineered for a 10-to-25-year wearable lifespan."
        )
    ))

    # ---------------------------------------------------------------------
    # DOSSIER 9: EXXONMOBIL CORP. (DEMONSTRATING TEMPORAL IRREVERSIBILITY
T_ir)
    # ---------------------------------------------------------------------
    dossiers.append(AuditDossier(
        dossier_id=9,
        name="ExxonMobil Corp. (US Shale Fracking, Extraction & Mega-Refining)",
        sector="Primary Fossil Energy Extraction, Hydraulic Fracturing & Petrochemicals",
        evaluation_unit="1 Hour of Integrated Extraction & Refining (~190,000 Barrels BOE/hr)",
        category=EntityCategory.OMEGA,
        supply_chain_tier=SupplyChainTier.TIER_1_EXTRACTOR,
        streams=[
            EmissionStream(
                name="Upstream Permian Shale Fracking (BTEX Slickwater, Radium Brine & Fugitive
Methane)",
```

```
            mass_tons=50.0,
            toxicity_df=28.0,
            vulnerability_ev=2.5  # Deep subterranean aquifers & marine shelves
        ),
        EmissionStream(
            name="Downstream Mega-Refining & Virgin Single-Use Plastic Monomer Synthesis",
            mass_tons=120.0,
            toxicity_df=10.0,
            vulnerability_ev=2.5  # Densely populated Gulf Coast littoral airsheds
        )
    ],
    lifecycle_lc=3.4,
    temporal_model=TemporalIrreversibilityModel(
        i_geo=1.5,              # Permanent shattering of deep shale & fossil aquifers
        c_bio=1.0,              # Delayed PAH/benzene cancer latency & microplastic trophic
cascade
        recovery_deficit_term=0.7   # 1,000+ yr aquifer renewal vs. instantaneous extraction
    ),                          # Total T_ir = 1.0 + 1.5 + 1.0 + 0.7 = 4.2
    manipulation_mf=5.0,
    annual_units_multiplier=8760.0,
    historical_units_multiplier=306600.0,  # 1977-2026 (since internal climate models verified
warming)
    reported_annual_profit_usd=40_000_000_000.0,
    conventional_fines_paid_usd="~$5M - $50M (while receiving billions in tax subsidies)",
    hypocrisy_rationale=(
        "Accurately predicted fossil-fuel warming in 1977 internal models, then funded "
        "40 years of public disinformation while promoting a <1% 'plastic recycling' myth."
    ),
    renewal_pact_solution=(
        "Pivot deep-drilling rigs from toxic shale fracking (T_ir = 4.2) to closed-loop "
        "supercritical geothermal wells (5-10 km depth, T_ir = 1.0) and end single-use plastics."
    )
  ))

  return dossiers


#
================================================================================
====
# 6. DEVELOPER EXTENSION API & REPORTING CLI
#
================================================================================
```

```
====

class SyndicateOracleEngine:
    """
    Main interface for developers to run audits, register custom corporate
    dossiers, simulate the Technological Renewal Pact, and export JSON data.
    """

    def __init__(self) -> None:
        self.dossiers: List[AuditDossier] = build_canonical_dossiers()

    def register_custom_dossier(self, dossier: AuditDossier) -> None:
        """Allows external developers to plug in new companies or local factories."""
        self.dossiers.append(dossier)

    def export_all_to_json(self, filepath: Optional[str] = None) -> str:
        """Exports the complete audit matrix and Renewal Pact simulations to JSON."""
        payload = []
        for d in self.dossiers:
            entry = asdict(d)
            entry["calculated_metrics"] = {
                "unadjusted_physical_ton_points": d.unadjusted_physical_ton_points,
                "lifecycle_adjusted_ton_points": d.lifecycle_adjusted_ton_points,
                "temporal_irreversibility_tir": d.temporal_model.coefficient,
                "temporal_adjusted_ton_points": d.temporal_adjusted_ton_points,
                "final_syndicate_ton_points": d.final_syndicate_ton_points,
                "unit_tariff_usd": d.unit_tariff_usd,
                "annual_liability_usd": d.annual_liability_usd,
                "cumulative_historical_debt_usd": d.cumulative_historical_debt_usd,
                "renewal_pact_simulation": d.simulate_renewal_pact(),
            }
            payload.append(entry)

        json_str = json.dumps(payload, indent=2)
        if filepath:
            with open(filepath, "w", encoding="utf-8") as f:
                f.write(json_str)
        return json_str

    def print_master_report(self) -> None:
        """Prints a formatted terminal report of all 9 canonical dossiers."""
        sep = "=" * 88
        print(sep)
```

```
        print("THE SYNDICATE: GLOBAL COMPENSATION MATRIX & EQUILIBRIUM AUDIT
ENGINE (ACT 2026)")
        print("Base Rate (B_r): $10.00 per Metric Ton-Point ($0.01 per kg-point)")
        print(sep)

        for d in self.dossiers:
            pact = d.simulate_renewal_pact()
            print(f"\n[{d.dossier_id}] {d.name.upper()}")
            print(f"    Sector:           {d.sector}")
            print(f"    Evaluation Unit:   {d.evaluation_unit}")
            print(f"    Supply-Chain Tier: {d.supply_chain_tier.value}")
            if d.one_time_capital_debt_usd > 0:
                print(f"    One-Time Hull Capital Debt: ${d.one_time_capital_debt_usd:,.2f}")
            print(f"    Physical Base:     {d.unadjusted_physical_ton_points:,.1f} ton-pts")
            print(f"    Lifecycle (L_c):   x{d.lifecycle_lc} -> {d.lifecycle_adjusted_ton_points:,.1f}
ton-pts")
            print(f"    Temporal (T_ir):   x{d.temporal_model.coefficient} ->
{d.temporal_adjusted_ton_points:,.1f} ton-pts")
            print(f"    Hypocrisy (M_f):   x{d.manipulation_mf} -> {d.final_syndicate_ton_points:,.1f}
final ton-pts")
            print(f"    ------------------------------------------------------------------")
            print(f"    >>> SYNDICATE UNIT TARIFF:      ${d.unit_tariff_usd:,.2f} / unit")
            print(f"    >>> ANNUAL CORPORATE LIABILITY: ${d.annual_liability_usd:,.2f} / year")
            if d.reported_annual_profit_usd:
                ratio = (d.annual_liability_usd / d.reported_annual_profit_usd) * 100.0
                print(f"    >>> REPORTED PAPER PROFIT:      ~${d.reported_annual_profit_usd:,.2f} /
year "
                      f"(Eco-Debt = {ratio:.1f}% of profit)")
            if d.cumulative_historical_debt_usd > 0:
                print(f"    >>> HISTORICAL PLANETARY DEBT:
${d.cumulative_historical_debt_usd:,.2f}")
            print(f"    >>> RENEWAL PACT TRANSITION:    Drops unit bill by
{pact['tariff_reduction_percentage']}% "
                  f"to ${pact['post_pact_unit_tariff_usd']:,.2f} & writes off 100% of historical debt.")
            print(f"    >>> Action Required:            {d.renewal_pact_solution}")

        print(f"\n{sep}")
        print("ENGINE VERIFICATION COMPLETE: ALL 9 DOSSIERS CONSERVE MASS &
DIMENSIONAL INTEGRITY.")
        print(sep)


if __name__ == "__main__":
```

```
engine = SyndicateOracleEngine()
engine.print_master_report()
```